

096224: Distributed Database Management

Spring 2025

Project Assignment 2

Background

It was a beautiful sunny day in Townsville. The Powerpuff Girls were having a picnic in the park, sitting on a pink blanket, eating frozen yogurt with pineapple and mango toppings, birds chirping peacefully around them, and kids laughing.

Choo choo. A pink train hit the Powerpuff Girls. They're gone (oh no... RIP).

Anyway... in this project you need to help some random cable company analyze their data and look for patterns in how different types of households watch TV. You'll explore demographic features, cluster households, and compare the viewing habits of different groups - both in static and streaming settings.

Several tasks lie ahead. Read the instructions carefully before you begin.

Data

The data in this project is a **variation** of the data used in the previous project.

Program Viewing Data

A variation of the viewing data you saw in project 1.

- `device_id` - Globally unique device identifier.
- `event_date` - Date viewing event occurred (Local) YYYYMMDD.
- `event_time` - Time viewing event occurred (Local) HHMMSS.
- `station_num` - Station ID number
- `prog_code` - Unique program identifier.
- `household_id` - The unique and persistent identifier for the household that the device belongs to.

Sub Demographic Data

Contains a wide variety of household characteristics information (a smaller version of the data you saw in project 1)

- `household_id` - The unique and persistent identifier for the household.
- `household_size` - Number of people residing in the household.
- `num_adults` - Number of adults residing in the household.
- `num_generations` - Number of Generations in Household.
- `marital_status` - Code indicating marital status of the head of household.
- `race_code` - Code to indicate the race of the head of household.
- `dwelling_type` - Dwelling type indicator code.
- `home_owner_status` - Home owner/renter code.
- `length_residence` - Length of time in years that the head of household has lived in the household.
- `home_market_value` - Value of the household.
- `net_worth` - Net worth of the household.
- `gender_individual` - Gender of the individual used to identify the household.
- `education_highest` - Code that indicates the highest level of education.

Assignment

Static Data Analysis (65 Points)

Feature Extraction

Your initial task involves examining and transforming the demographic dataset to prepare it for analysis.

Begin by preprocessing and normalizing the demographic data attributes:

1. Numerical columns: Scale each column according to the maximum and minimum values in that column so each value is in the $[0, 1]$ range, use $\frac{x-\min}{\max-\min}$.
2. Categorical columns: Apply one hot encoding to change the values of the column to a binary vector representation.

Then, combine all processed columns into a single feature vector representing each household. **Note:** `household_id` is an identifier and not to be included in the features vector.

Show 7 rows from the resulting DataFrame.

Answer in your PDF Report:

- Why do we normalize numerical columns?
- Why do we use one-hot encoding instead of assigning numerical indices to the categories?

Visual Analysis

Now that we have one column that contains all our useful information, we can finally apply some lovely algorithms!

Let's check if the households naturally group into clusters using PCA (yay!).

Use PySpark's built-in PCA (from MLlib) with $k = 2$ to project the feature vectors onto $\mathbb{R}^2$, then create a scatter plot - make sure the scatter plot points are **pink**.

To plot, you can use: `pca_df.toPandas().plot.scatter(x='x_col_name', y='y_col_name')`

Show 7 rows from the resulting DataFrame and **display** the scatter plot.

Answer in your PDF Report:

- What do you observe in the scatter plot? How many clusters do you think are visible? We will denote the number of clusters you observed with c .
- Attach the scatter plot.

Clustering

The PCA algorithm can give us a rough intuition about clusters, however, it's not a clustering algorithm! To actually divide the data into groups, we need to use an algorithm like K-means!

Use PySpark's built-in K-means (from MLlib), with $k = c$ and seed set to 3 (the seed is 3 cause that's how many Powerpuff Girls there used to be lol).

Then:

- Add a column assigning each `household_id` to a cluster (cluster names don't matter).
- Add a column with the distance of each household from its cluster's centroid. Think which tool we learned can be used to implement this easily.
- **Show** 7 rows from the resulting DataFrame.

Dividing households into subsets

In this subsection, you must use PySpark Window functions for all subset calculations.

Start by ordering households within each cluster by their distance to the cluster's centroid (ascending - closest first).

For each cluster:

- Extract every **3rd** household (i.e., 3rd, 6th, 9th, ...) - we'll call this the *3rds subset*.
- Extract every **17th** household (17th, 34th, ...) - we'll call this the *17ths subset*.
- Keep the full list of households - we'll call this the *Full* subset.

You should now have **3 subsets per cluster** (Full, 3rds, and 17ths), for a total of **3c subsets across all c clusters**.

Important: The clustering and subset selection is performed **once**. Any reference to these subsets later refers back to this original partitioning. Each `household_id` is assigned to one cluster only, and may belong to 1, 2, or all 3 subsets of that cluster (e.g., a household ranked 51st in its cluster would appear in all 3 subsets: Full, 3rds, and 17ths).

Cluster's Viewing Analysis

Next, we want to assess whether the demographic clustering we've done has any real-world meaning. To do this, we'll use the viewing data to check whether households in different clusters actually watch different types of content.

We'll compute a measure called `diff_rank` for each station within each cluster (and its subsets), following this process:

1. For each **subset of each cluster**, count the number of viewing events (rows in `viewing_data`) per `station_num`.
2. Convert those counts into percentages: divide each station's count by the total number of viewings in that subset, then multiply by 100. This gives you the **popularity rating** of each station in that subset.
3. Repeat steps 1 and 2 for the **entire dataset** (i.e., without clustering). This gives the **general population's** popularity rating per station.
4. For each station in each subset, subtract the general population rating from the subset's rating:

$$\text{diff_rank} = \text{subset_rating} - \text{general_rating}$$

This is how much more or less popular the station is in the subset compared to the average household.

Then:

- For **each subset**, return the top 7 stations with the highest `diff_rank`. You should end up with $3c$ sets — 3 subsets per cluster.
- Show the results clearly, with proper titles to distinguish between subsets (e.g., "Cluster 2 — 17ths Subset").
- Each cluster's 3 result sets should be presented close together for comparison.
- You may take creative freedom with how you present the data, as long as clarity is maintained.

Answer in your PDF Report:

- What does a positive or negative `diff_rank` mean for a station? What insights can it give us about the viewing preferences of a specific cluster compared to the general population?
- Compare the same subset types (Full, 3rds, 17ths) across clusters, and also compare different subset types within the same cluster. Do we observe meaningful differences? Does the clustering appear to reflect actual differences in content preference?

Dynamic Data Analysis - Streaming (35 Points)

Use Spark Streaming to extract viewing data from Kafka, and repeat the *Cluster's Viewing Analysis* task - this time on *streaming data*.

Instructions:

- Use the starter code from the notebook to connect to the Kafka stream and begin reading data.
- For each trigger, perform the viewing analysis using only the 3rds subset per cluster. That is, for each trigger you should output *one group* of top stations per cluster - not all three.

Implementation Requirements:

- Process and show results for at least **3 batches** (triggers).
- The starter code does **not** handle batch counting - you need to implement this yourself.
- You must show the output for each batch separately. If outputs don't appear in the notebook, check the **Driver Logs** (via your running cluster menu) and copy them manually into the appropriate notebook cell.

Important Constraints (and hints):

- **You must not use the static viewing data in this part** - only the data being streamed.
- However, you **can** (and should) use the demographic clusters and 3rds subset dataframes and logic you have from previous tasks.
- Results from each batch should be **incremental** - each output should include data from all previous batches (think, what does this mean?)
- Think carefully about how you aggregate data. Is collecting raw viewing entries scalable? Will it slow down streaming in the long run?

Answer in your PDF Report:

- Compare the results across clusters for each batch.
- Look for recurring patterns - are clusters consistently drawn to similar stations?
- Is there an observable connection between households in the same cluster and their preferred stations?

Competitive Bonus (2 points)

In your PDF report, guess your TAs' favorites in the following format:

- Anis: (one of Anis' favorite frozen yogurt toppings)
- Aviv: (Aviv's favorite color)

Additionally, each group member should write a 2-3 line paragraph about:

- Member 1: Your favorite type of frozen yogurt (cow milk/soy/goat milk/etc.) and toppings.
- Member 2: Your favorite color and why.
- Member 3 (if applicable): What color you'd make all frozen yogurt world-wide.

You get one point per correct guess. Answers will be revealed sometime after the project's submission.

Good Luck!

**Course Staff, the daughter of the CEO of that random cable company,
and whoever was operating that train**

General and Submission Guidelines

General Guidelines

- The project must be done **exclusively** on Databricks.
- Use **only PySpark** and `matplotlib` (when plotting is needed).
- Write clean, readable code and document functions where necessary.
- All project-related questions will be answered in the Moodle forum, **exclusively**.
- Only one team member needs to submit the project.
- Use `display(df.limit(n))` to show DataFrames when asked. Alternatively, `df.show(n, truncate=False)` is acceptable if it looks better.
- Submission deadline: **August 13th at 23:59**.
- Use of generative AI tools (e.g., ChatGPT) is allowed, but we recommend using them sparingly and only after trying to solve the problem yourself. If used, briefly note where and how it helped. This will **not** affect your grade.
- Pretty and visually appealing assignments will be graded more favorably :)

Submission Guidelines

1. **PROJECT2_WET-[ID1]-[ID2].ipynb**: A **Jupyter Notebook** containing your code, analysis, explanations, and outputs.
2. **PROJECT2_WET-[ID1]-[ID2].pdf**: A **PDF** version of your notebook, with outputs included.
3. **PROJECT2_WET-[ID1]-[ID2].html**: An **HTML** version of your notebook, with outputs included.
4. **PROJECT2_DRY-[ID1]-[ID2].pdf**: A **PDF** containing your analysis and **answers to all written questions** throughout the instructions.

Replace [ID1], [ID2] (and [ID3] if needed) with the IDs of the students in your group.